The Determinism Contract: What CKS Substrates Must Guarantee About Reproducibility, and What Breaks It

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 24 April 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, as a single named contract, the determinism and reproducibility guarantees the source paper distributes across several sections, so that downstream implementers and auditors can refer to those guarantees as a single citable specification of what any CKS-coherent substrate must satisfy.

Abstract

The CKS pattern commits, in several places, to properties that together amount to a determinism and reproducibility contract over the substrate. The commitment surfaces in §2.1 (substrate as inspectable artifact), §3.1 (traceability, auditability, conflict preservation, and governance as constitutive design goals), §4.1 (the substrate is "deterministic, human-governed, auditable" while the LLM handles high-dimensional reasoning), §6.2 (the *context rot* framing that motivates external structured memory), §8.2 (the deterministic-substrate-plus-LLM split referenced via KGLM), and §11.3 (the substrate as the source of truth for what was decided, by whom, under what authority, with what rationale). The source paper does not collect these commitments under a single name. This note does. It identifies the two-layer scope of the contract — determinism at the substrate (representation) layer, not at the LLM-output layer — states the five guarantees the contract makes, names the non-determinism the contract permits, identifies five anti-patterns that violate it, and gives an operational test by which any system can be checked for contract coherence at any time during the substrate's existence.

1. Why the contract needs to be stated

The CKS pattern's determinism commitment is real, defended, and load-bearing — but it is distributed across the source paper rather than collected under a single name. The abstract above lists the six load-bearing sections; each defends one piece of the commitment, and none is named as part of a single contract. The substrate's role as inspectable artifact (§2.1), as carrier of traceability and audit (§3.1), as the deterministic side of the governance boundary (§4.1), as the antidote to *context rot* (§6.2), as the deterministic-provenance side of the KGLM split (§8.2), and as source of truth for what was decided (§11.3) are six commitments that together imply a contract — but the source paper does not collect them as one.

That is what this note adds. Naming the commitment as a contract — a set of guarantees the substrate makes to its readers, writers, and downstream consumers — gives downstream implementers a single reference for what they must satisfy, gives auditors a single reference for what they must verify, and gives critics a single target to argue with. The contract introduces no commitment the source paper has not already made; it only collects them.

2. Two layers — representation determinism versus model-output determinism

CKS's determinism commitment applies to one of two architectural layers and not to the other. Distinguishing them is the most important framing decision in this note, because the contract's scope follows directly from the distinction.

Representation layer (the substrate). This is the layer the contract binds. The substrate is a structured artifact carrying entities, relationships, decisions, rationale, and conflicts — human-governed, inspectable, persistent across sessions. The commitments §4.1 attaches to it are explicit: the substrate is *deterministic, human-governed, auditable*. Determinism here means substrate state is well-defined at every moment, that the same state yields the same answers when read, that changes to state are addressable, and that no non-deterministic process silently changes what the substrate carries.

Model-output layer (the LLM). This is the layer the contract does *not* bind. LLMs are non-deterministic by nature of the model: temperature, sampling, and model-internal state mean that the same input may yield different outputs across runs. §4.1's governance-boundary phrasing acknowledges this directly — the LLM is the side of the boundary that handles high-dimensional reasoning, and that side is not architected to be deterministic. §8.2's KGLM reference has the same shape: the substrate side is deterministic; the LLM side is the verbalizer.

The contract binds the substrate. LLMs operate over the substrate, but they are not themselves bound by it. What the contract requires of LLMs is indirect: anything an LLM writes back to the substrate is bound by the contract from the moment of the write, and any cell-level behavior determined by orchestration rules over substrate state is bound at the substrate-write layer (what gets recorded), not at the LLM-output layer (what intermediate text the model produced to get there). Misreading the scope in either direction breaks the architecture: binding LLM outputs makes the contract impossible to satisfy and forecloses the AI-as-substrate-mediator role; binding nothing in particular makes the substrate's commitments evaporate. The contract binds the representation; the model is out of scope.

3. The reproducibility contract — five guarantees

A CKS-coherent substrate satisfies, at all times, the following five guarantees. Each is derived from a specific commitment in the source paper; each binds the representation layer; none extends to LLM outputs.

(a) Same substrate state yields the same substrate-mediated read content. Two reads of identical substrate state — by any reader, at any time, through any cell that supports reading — must yield identical content. The substrate's job under §2.1 is to be inspectable; under §11.3 it is to be the source of truth for what was decided. Both commitments require that what the substrate carries is well-defined and that querying it returns that content reproducibly. The guarantee binds the answer the substrate

gives, not the natural-language wrapping a particular cell or LLM may put around it.

(b) Same substrate state plus same orchestration rules yields equivalent cell-level behavior, modulo non-deterministic LLM outputs. Two cells executing under the same orchestration rules over the same substrate state must produce equivalent decisions. Equivalence here is judged at the substrate-write layer — what gets written back to the substrate, what state changes are committed, which orchestration rule was selected — not at the LLM-output layer. This is the cell-level reading of §4.1's governance boundary: cell behavior insofar as it touches substrate state inherits its determinism from the substrate and the rules, not from the LLM.

(c) Substrate state changes are addressable. Every change to substrate content has a locatable origin: who or what wrote it, under what orchestration rule, and with what rationale where rationale is part of the substrate's content type. This is the path-retraceability commitment §3.1 imports from Rajabi and Kafaie (2022) and applies to substrate writes. Addressability is what makes the substrate audit-bearing rather than merely persistent, and it is the property §3.1's accountability-trace vocabulary names.

(d) Conflict states are preserved, not silently collapsed. Two contradictory pieces of substrate content remain contradictory until human authority — exercised directly by editing the substrate, or indirectly through human-authored orchestration rules that specify a resolution under specified conditions — resolves them. No non-deterministic process selects between contradictions on the substrate's behalf. This is §5's conflict-preservation commitment restated as a contract guarantee: contradictions cannot be erased by an LLM operation under any delegation level, and resolution, when it occurs, is recorded as a resolution decision rather than as a silent overwrite.

(e) Substrate state is the source of truth. Any decision the system makes about *what is currently the case* — what has been decided, what role holds what authority, which contradictions are open, which are resolved and how — must be answerable from the substrate, not from LLM context, not from agent memory, not from per-session storage, not from any state held outside the substrate. This is the §11.3 source-of-truth commitment generalized: the substrate is the artifact the system stands behind, and any answer derived elsewhere is non-substrate state and outside the contract's coverage.

The five guarantees are independent — a system can satisfy any subset and fail others — and CKS coherence requires all five.

4. Allowed non-determinism

The contract does not require everything about a CKS deployment to be deterministic. Several kinds of non-determinism are admissible and do not violate any of the five guarantees.

LLM output text. The LLM is non-deterministic by design. Two runs of the same prompt may produce different intermediate text, phrasing, or ordering. The contract binds the substrate, not the model; what constrains the LLM is what gets written back to the substrate under the orchestration rules.

Timestamps and rendering metadata. Substrate content may be presented with current-time stamps, environment-specific rendering, or display-layer formatting that varies across views without the underlying substrate state changing. Presentation metadata is a property of the rendering, not of the substrate.

Presentation order. Equivalent substrate state may be displayed in different orders by different cells without violating the contract, provided the underlying state is identical. Order is a presentation property; the substrate's content is what the contract binds.

Cell execution paths. Different cells operating over the same substrate may take different execution paths under their own orchestration rules. What the contract binds is what they write back to the substrate, not the path they took to arrive there. Path differences that produce equivalent substrate writes are within the contract.

The pattern is the same in each case: the contract binds substrate state, the determinism of reads against it, and the addressability of changes to it; anything outside the substrate is outside the contract.

5. What breaks the contract — anti-patterns

Five anti-patterns recur across systems that approximate CKS without satisfying the contract. Each is named below with the guarantees it violates.

Hidden state. Substrate-relevant state held in LLM context, agent memory, or session storage rather than in the substrate. Violates (e): if the answer to *what is currently the case* lives outside the substrate, the substrate is no longer the source of truth, and the system is one session expiry away from losing what it depended on. This is the failure mode §6.2 names as *context rot*, applied to coordination state rather than only to facts.

Implicit context. Cell behavior depends on context that is not represented in the substrate or in orchestration rules. Violates (b), because two cells with the same substrate and the same rules may behave differently if one is silently consulting context the substrate does not carry; also violates (c), because the implicit-context source has no addressable origin.

Agent memory as source of truth. When *what is currently the case* is answered from per-session or external agent memory rather than from the substrate. Violates (e). Same shape of failure as hidden state, but with an additional architectural commitment behind it: the system has chosen agent memory as a substitute for substrate, not merely as a cache. The §6.2 separation-of-concerns commitment is exactly what this anti-pattern abandons.

Silent conflict resolution. When contradictions between substrate content are collapsed by an LLM, or by an automated process, without being recorded as a resolution decision. Violates (d). The substrate may end up internally consistent — which can look like correct behavior — but the resolution path is unrecoverable. Resolution by human authority, or by a human-authored orchestration rule that specifies a resolution under specified conditions, is admissible and recorded; silent resolution is not.

Non-addressable writes. Substrate content that exists but cannot be traced to a writer, rule, or rationale. Violates (c). Non-addressable writes appear most often when an automated pipeline appends to the substrate without recording its own provenance, or when an LLM is permitted to modify substrate state without the orchestration rules requiring an addressable origin. Addressability is not a logging convention bolted on later; under the contract, it is a property the substrate must carry from the moment of the write.

In each of these anti-patterns the system may still be useful for some purpose. What it loses is contract coherence. A substrate that exhibits any of the five is not CKS-coherent in the sense the source paper defines, and downstream consumers cannot rely on its guarantees regardless of how reliably it appears to behave in any given session.

6. Operational test

Regression-testing a CKS-coherent system against the contract is the operational application of the five guarantees. The contract is the specification; any test that verifies a system continues to satisfy it over time — under change in the substrate, the orchestration rules, the LLM, or the deployment environment — is, by construction, a test of the five guarantees. This note does not propose a specific testing framework or methodology; doing so would introduce commitments the source paper does not make. It names the invariants any test must verify and leaves the test mechanism open, in the same way the human-governed note's operational test names the rights any check must verify and leaves the check mechanism open.

A system satisfies the determinism contract if and only if all of the following are true at all times during the substrate's existence:

1. **Read determinism.** Two reads of identical substrate state, by any reader, through any cell that supports reading, yield identical content.

2. **Write determinism modulo LLM.** Two cells executing under the same orchestration rules over the same substrate state produce equivalent substrate writes — equivalence judged at the substrate-write layer, not at the LLM-output layer.

3. **Write addressability.** Every change to substrate content has a locatable origin: writer, orchestration rule, and rationale where rationale is part of the substrate's content type.

4. **Conflict preservation.** Contradictions in substrate content are not collapsed by any non-deterministic process; resolution, when it occurs, is the result of human authority or of a human-authored orchestration rule and is recorded as a resolution decision.

5. **Substrate as source of truth.** Any answer to *what is currently the case* is derivable from the substrate, not from LLM context, agent memory, session storage, or any other state held outside the substrate.

A system that fails any of (1)–(5) may be a useful system, and may have other valuable properties, but is not contract-coherent in the CKS sense.

7. Why naming this contract matters

The determinism contract is not a new architectural commitment. It is a name for commitments the source paper has already made — distributed across §2.1, §3.1, §4.1, §6.2, §8.2, and §11.3 — collected so that they can be cited, tested, and argued with as a unit.

Naming the contract has three consequences. First, implementations of CKS that explicitly satisfy the contract are interoperable in a specific sense: downstream consumers can rely on the same five guarantees regardless of which implementation produced the substrate content. Two CKS-coherent

substrates may differ in tooling, schema, orchestration rules, and the LLM that mediates over them, while remaining substitutable at the contract layer. Second, implementations that do not satisfy the contract should not be called CKS-coherent. They may be useful for other purposes, but calling them CKS dilutes the term and loads downstream consumers with risks the architecture is committed to mitigate. Third, the contract gives critics of CKS a single, citable target. A reasoned argument that one of the five guarantees is misspecified, or that the two-layer scope is drawn at the wrong place, is an argument with the contract — and through it, with the source paper's commitments — that can be conducted in shared terms.

This note does not argue that determinism at the representation layer is universally desirable. There are systems for which non-deterministic representations are appropriate, and there are coordination problems for which the CKS pattern is not the right architecture. What this note argues is narrower: any system called CKS must guarantee what the contract specifies. Subsequent work that satisfies the source paper's other commitments without satisfying the contract is using the term CKS for something the source paper does not defend, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Determinism Contract: What CKS Substrates Must Guarantee About Reproducibility, and What Breaks It.* 24 April 2026. ORCID: 0009-0004-8065-3235.